Технологии конструирования программного обеспечения

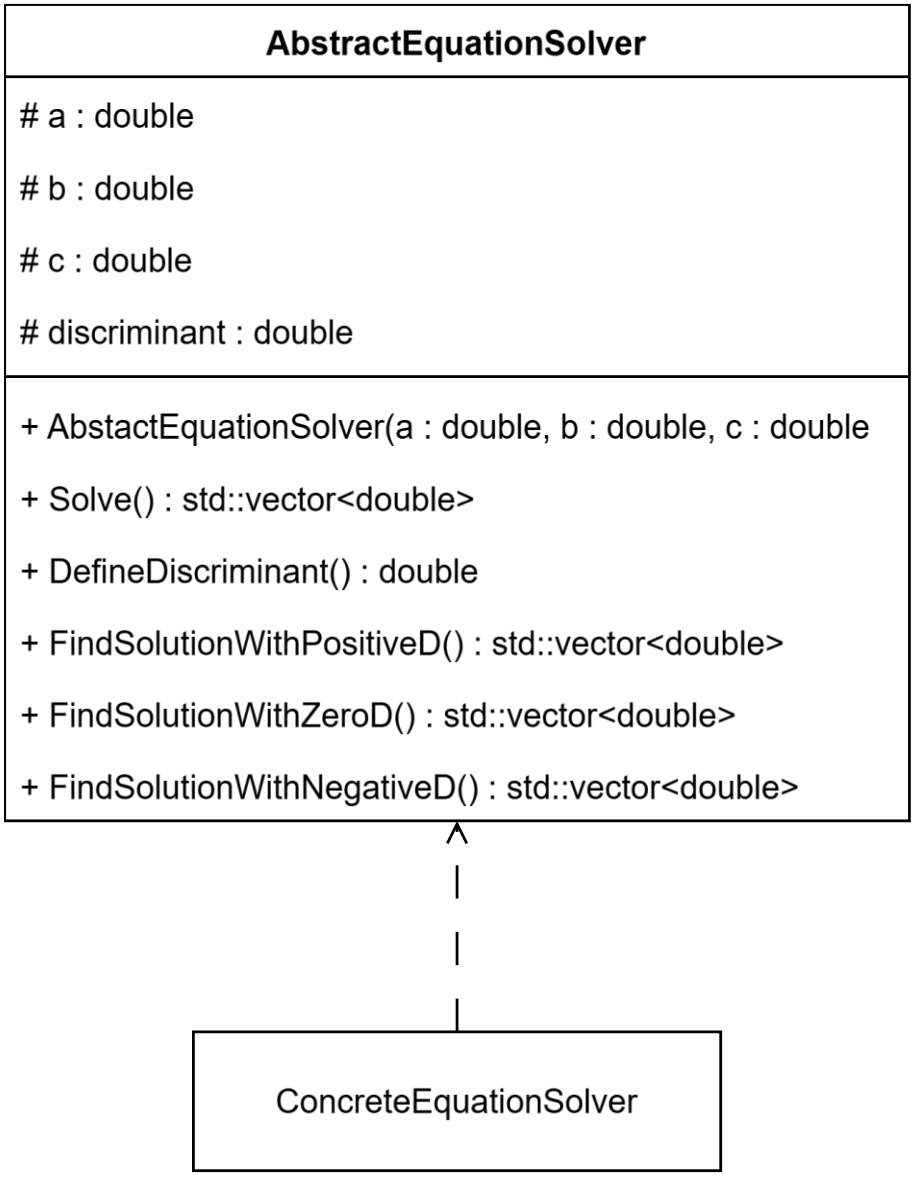
Отчет по экзаменационной работе № 2

Группа: 221-3210 **Студент:** Обухов Алексей Сергеевич

Задание на экзаменационную работу

Паттерн «Шаблонный Метод». Применить паттерн при написании программы решения квадратного уравнения. Шаги алгоритма должны меняться в зависимости от значения дискриминанта (положительный, отрицательный, равен нулю).

Диаграмма классов



Исходный код программы

Содержимое «TemplateMethod.h»

```
	#pragma once

#include <vector>

// Абстрактный класс решателя уравнений
class AbstractEquationSolver {
protected:
	double a, b, c;
	double discriminant;
public:
	AbstractEquationSolver(double a, double b, double c) {
		this->a = a;
		this->b = b;
		this->c = c;
	}

	// Шаблонный метод для решения уравнения
	std::vector<double> Solve() {
		discriminant = DefineDiscriminant();

		if (discriminant > 0)
			return FindSolutionWithPositiveD();

		if (discriminant == 0)
			return FindSolutionWitZeroD();

		return FindSolutionWitNegativeD();
	}

	// Шаг определния дискриминанта
	virtual double DefineDiscriminant() {
		return b * b - (4 * a * c);
	}

	// Решение, в случае если D > 0
	virtual std::vector<double> FindSolutionWithPositiveD() = 0;

	// Решение, в случае если D = 0
	virtual std::vector<double> FindSolutionWitZeroD() = 0;

	// Решение, в случае если D < 0
	virtual std::vector<double> FindSolutionWitNegativeD() = 0;

};

// Конкретный класс для решения уравнений
class ConcreteEquationSolver : public AbstractEquationSolver {
public:
	ConcreteEquationSolver(double a, double b, double c) :
AbstractEquationSolver(a, b, c) {}

	// Решение, в случае если D > 0
	virtual std::vector<double> FindSolutionWithPositiveD() override {
		// квадратный корень из дискриминанта
		double sqrtD = sqrt(discriminant);

		double x1 = (0 - b + sqrtD) / (2 * a);
		double x2 = (0 - b - sqrtD) / (2 * a);

		std::vector<double> solution;
		solution.push_back(x1);
```

```
            solution.push_back(x2);

            return solution;
      }

      // Решение, в случае если D = 0
      virtual std::vector<double> FindSolutionWitZeroD() override {
            double x1 = (-b) / (2 * a);

            std::vector<double> solution;

            // x1 добавляется в решение, поскольку при D = 0
            // уравнение имеет два равных корня
            solution.push_back(x1);
            solution.push_back(x1);

            return solution;
      }

      // Решение, в случае если D < 0
      virtual std::vector<double> FindSolutionWitNegativeD() override {
            // Возвращаем пустой вектор, поскольку решений нет (в вещественных
числах)
            return std::vector<double>{};
      }
};
```

Содержимое «main.cpp»

```
#include <iostream>
#include "TemplateMethod.h"

// Выполнил студент группы 221-3210 Обухов Алексей

/* Задание:
*     Паттерн «Шаблонный Метод».
*     Применить паттерн при написании программы решения квадратного уравнения.
*     Шаги алгоритма должны меняться в зависимости от значения дискриминанта
*     (положительный, отрицательный, равен нулю).
*/


void SolveEquation(double a, double b, double c) {
      ConcreteEquationSolver* solver = new ConcreteEquationSolver(a, b, c);

      std::vector<double> roots = solver->Solve();

      std::cout << "Roots of the equation:\n";

      if (roots.size() == 0)
            std::cout << "This equation has no roots.\n";
      else {
            std::cout << "This equation has " << roots.size() << " roots: ";

            for (int i = 0; i < roots.size(); i++) {
                  std::cout << roots[i] << " ";
            }
            std::cout << "\n";
      }

      delete solver;
```

```
	return;
}

int main() {
	double a, b, c;

	std::cout << "Enter A >> ";
	std::cin >> a;
	std::cout << "Enter B >> ";
	std::cin >> b;
	std::cout << "Enter C >> ";
	std::cin >> c;

	SolveEquation(a, b, c);

	return 0;
}
```